

Semana 4



Prácticas

PROGRAMACIÓN III

Depto. de Informática y Automática
Grado en Ingeniería Informática
Álvaro Lozano Murciego

Objetivos

1. Solventar posibles dudas de la sesión anterior (en caso contrario por correo)
2. Conocer el mecanismo para **proporcionar argumentos a una aplicación en java** desde el exterior. Ejemplo con un **.jar y desde el IDE**.
3. Conocer la diferencia entre **tipos primitivos y tipos de referencia**.
4. **Arrays en Java**: uso, funciones útiles, nuevo for (*for each*) y problemáticas asociadas.
5. Diferencia entre **método/atributo de instancia o de clase**. Uso de **static**.
6. Conocer el **Paradigma Orientado a Objetos**: cambio de paradigma.
7. Trabajar ejercicios: recuerda que las 2 horas de prácticas necesitan de 2 horas de trabajo personal en casa con los ejercicios.

*Es vital hacer todos los ejercicios **individualmente** desde cero, cometer errores y aprender de estos últimos.*

Contenidos

1. ¿Dudas hasta ahora?
2. Proporcionar argumentos por línea de comandos desde IDE 
3. Tipos primitivos vs Tipos de Referencia 
4. Arrays en Java: creación, asignación y atributos útiles 
5. Trabajar uso de static con un ejemplo: métodos y atributos 
6. Emplear clases y objetos: **cambio de paradigma.** 
7. Continuar con los ejercicios para la sesión 

*Es vital hacer todos los ejercicios **individualmente** desde cero, cometer errores y aprender de estos últimos.*

1. Dudas

¿DUDAS?

2. Argumentos por consola Java

Argumentos

```
public static void main(String[] args)
```

2. Argumentos por consola (VSCode)

The image shows a VS Code window with the Run and Debug panel on the left and the Explorer and Editor panels on the right. The Run and Debug panel has a 'Run and Debug' button and instructions to create a launch.json file. A red arrow labeled '1' points to the 'Run and Debug' button. The Explorer panel shows the file structure of a project named 'NUEVOPROYECTOJAVA', with the '.vscode' folder expanded, showing 'launch.json' and 'settings.json'. The Editor panel shows the 'launch.json' file with two configurations. A red arrow labeled 'Configuración' points to the first configuration, and another red arrow labeled 'Configuración' points to the second configuration. A text box on the right explains that the file contains various execution configurations and that arguments will be added to one of them using the 'args' key.

1

En primer lugar, crearemos un fichero launch.json de configuración de ejecución para VSCode si aún no lo habíamos hecho.

Directorio:
.vscode/launch.json

Configuración

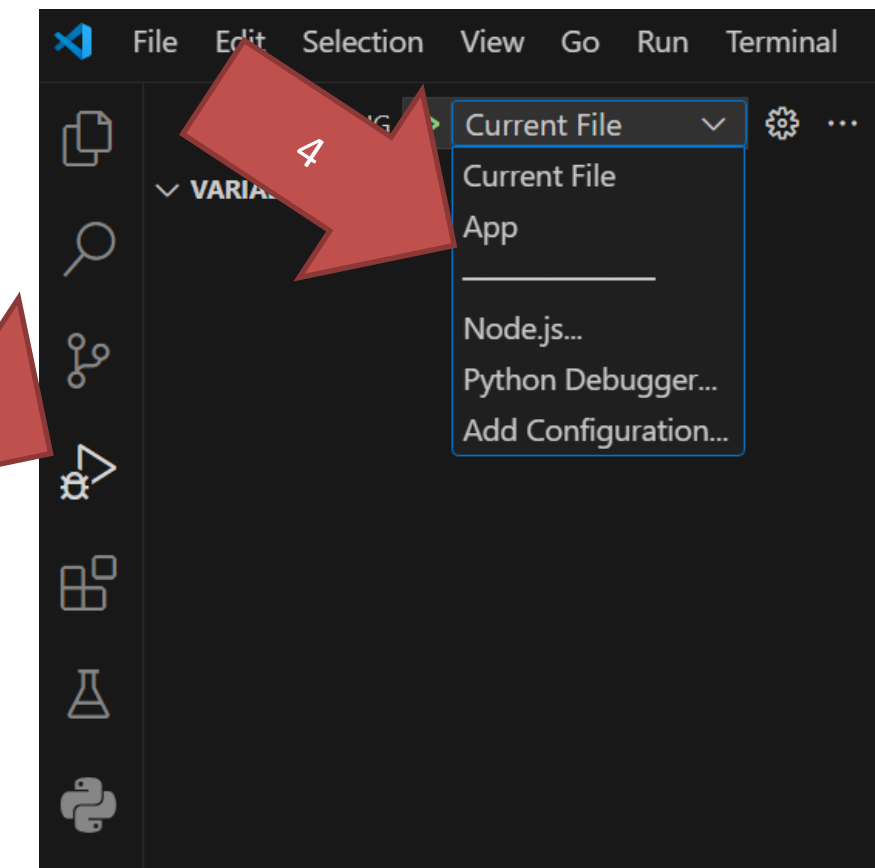
Configuración

Aquí se ve un archivo con varias configuraciones de ejecución. Será en una de ellas en la que añadiremos los parámetros con la clave "args"

```
.vscode > {} launch.json > ...
1  {
2      // Use IntelliSense to learn about possible attributes.
3      // Hover to view descriptions of existing attributes.
4      // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
5      "version": "0.2.0",
6      "configurations": [
7          {
8              "type": "java",
9              "name": "Current File",
10             "request": "launch",
11             "mainClass": "${file}"
12         },
13         {
14             "type": "java",
15             "name": "App",
16             "request": "launch",
17             "mainClass": "App",
18             "projectName": "NuevoProyectoJava_c9e97046"
19         }
20     ]
21 }
```

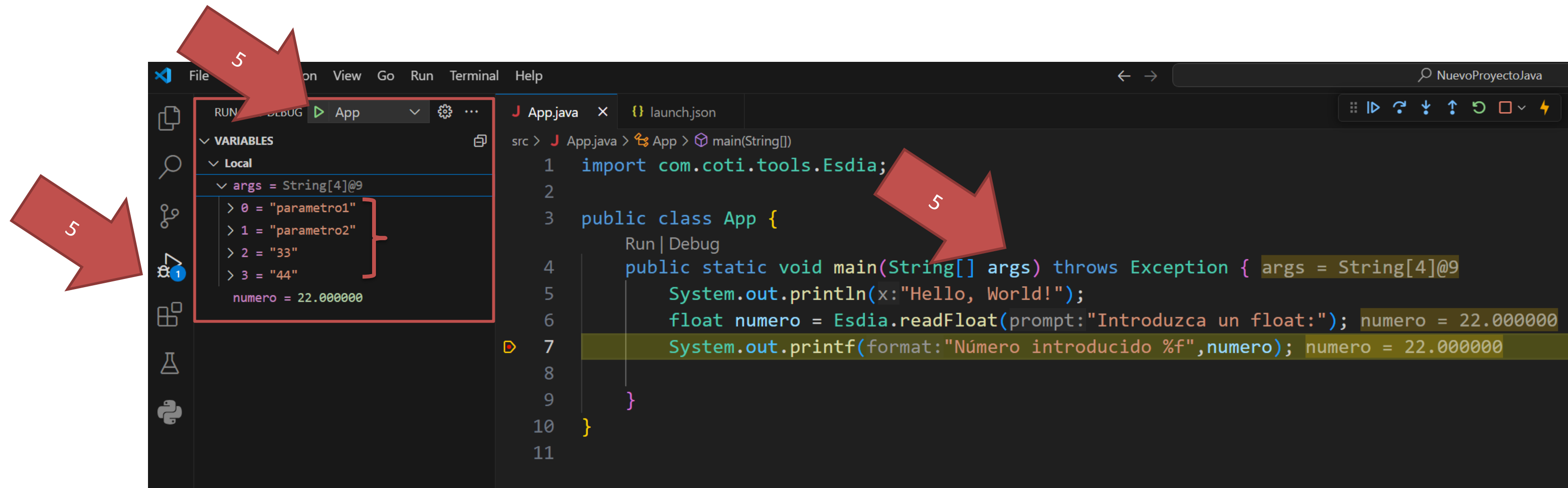
2. Argumentos por consola (VSCode)

```
.vscode > {} launch.json > Launch Targets > {} App
1  {
2      // Use IntelliSense to learn about possible attributes.
3      // Hover to view descriptions of existing attributes.
4      // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
5      "version": "0.2.0",
6      "configurations": [
7          {
8              "type": "java",
9              "name": "Current File",
10             "request": "launch",
11             "mainClass": "${file}"
12         },
13         {
14             "type": "java",
15             "name": "App",
16             "request": "launch",
17             "mainClass": "App", // Clase con método main que se va a ejecutar
18             "projectName": "NuevoProyectoJava_c9e97046",
19             "args": "parametro1 parametro2 33 44"
20         }
21     ]
22 }
```



Parámetros que se pasan a como Array de String. Tened en cuenta que se indican separados por espacios y son recibidos como cadenas del array de Strings **args**

2. Argumentos por consola (VSCode)



Aquí se ha establecido un punto de interrupción, se ha lanzado con la configuración llamada “App” y es posible ver el Array de String args **con los cuatro valores indicados en el fichero launch.json** anteriormente

Parámetros por la línea de órdenes



¡SON STRINGS!

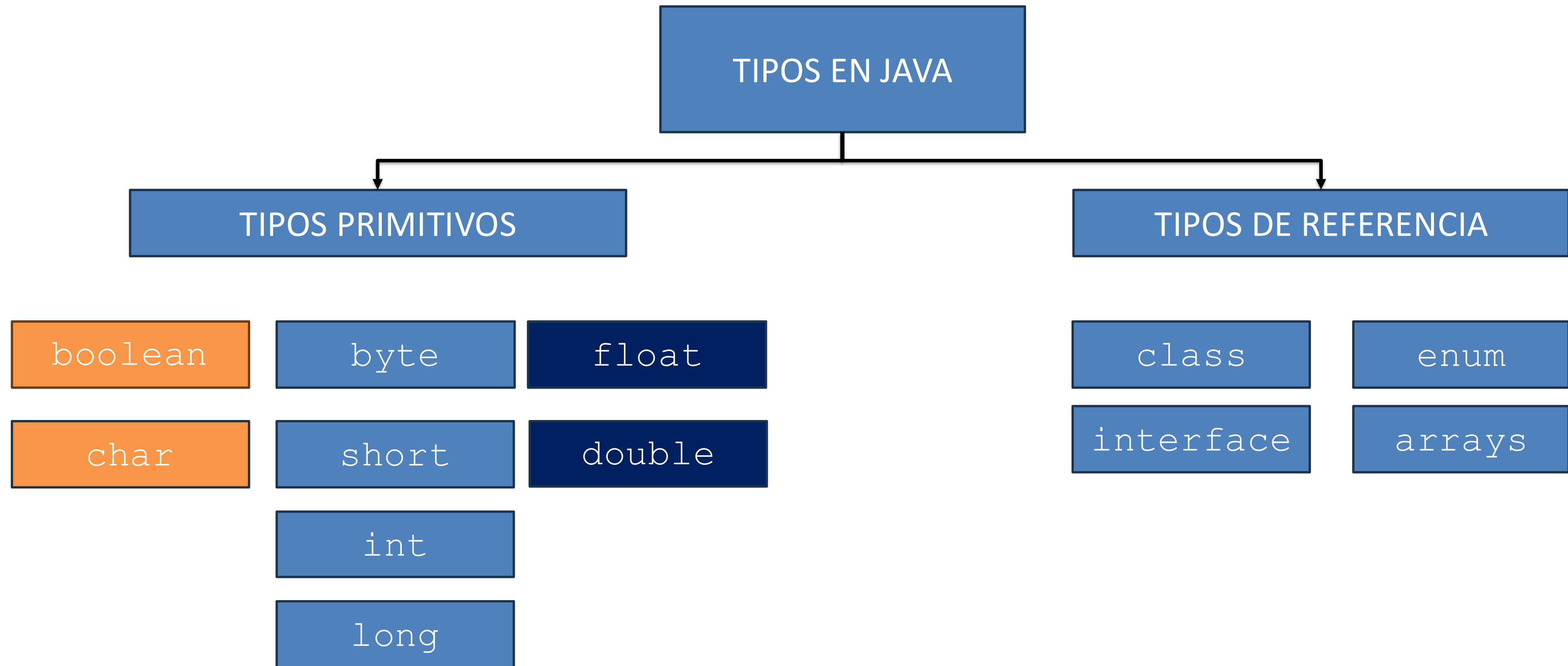
`java -jar program.jar param1 param2 ...`

¿Y si quiero pasar una
cadena con espacios?
¡Comillas dobles!

`java -jar program.jar "hola mundo"`

`java paquete.MiClase param1 param2 ...`

3. Tipos primitivos vs Tipos de referencia

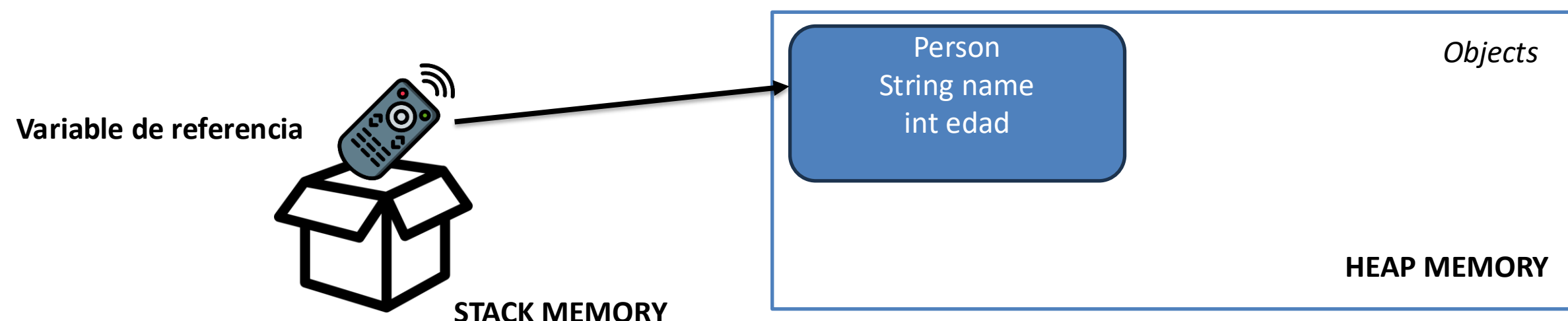


3. Tipos primitivos vs Tipos de referencia

Tipo primitivo en Java: se trata de un tipo de variable que puede almacenar un único **valor** en un determinado momento. E.g. una variable int puede contener un valor determinado e.g. 33. Si se asigna otro valor, reemplazará el que había antes.



Tipo de referencia: también llamados referencias, albergan la **localización** de un objeto. Estas variables “**se refieren**” a un **objeto** en el programa. Una buena analogía es pensar en ellas como un mando a distancia del objeto que está en memoria (en la HEAP). Por ejemplo, la referencia que se obtiene cuando se crea un String o cualquier objeto de una clase que hayamos definido. Ya que hace referencia a un objeto, tendremos acceso a métodos y atributos del mismo.



3. Tipos primitivos vs Tipos de referencia

IMPORTANTE: Aspectos a tener en cuenta (realizar un ejemplo con cada uno)

TIPOS PRIMITIVOS	TIPOS DE REFERENCIA
Almacenan valores . Si no se le asigna ningún valor, tomarán un valor por defecto dependiendo del tipo, e.g. float con 0.0f (si son atributos de una clase). Las variables locales no tienen valor por defecto (hay que inicializarlas).	Almacenan manejadores de objetos. Si no se le asigna ningún valor tomarán el valor null . (si son atributos de una clase). Las variables locales no tienen valor por defecto (hay que inicializarlas).
Asignación (=): cuando se asigna un valor a un tipo primitivo, <u>el valor es copiado</u> .	Asignación (=): cuando se asigna una referencia a otra, dicho manejador es copiado . Cuidado aquí, porque ahora tendríamos 2 “mandos” que apuntan al mismo objeto.
Consecuencia del punto anterior cuando los métodos tienen tipos primitivos como parámetros: El valor es copiado, por lo que la variable original no se modifica ya que el parámetro es otra variable .	Consecuencia del punto anterior cuando los métodos tienen tipos de referencia como parámetros: El valor del manejador es copiado, por lo que el objeto original al que apunta podría modificarse (“si utilizamos el mando”, es decir, si modificamos los atributos del objeto al que apunta la referencia) .

3. Tipos primitivos vs Tipos de referencia

IMPORTANTE: Aspectos a tener en cuenta (realizar un ejemplo con cada uno)

TIPOS PRIMITIVOS	TIPOS REFERENCIA
Operador de comparación (==): cuando dos variables de tipo primitivo se comparan con este operador, sus valores primitivos son comparados .	Operador de comparación (==): cuando dos variables de tipo de referencia se comparan con este operador, compara identidad (la misma referencia) . Veremos más adelante un ejemplo con String y cómo solventarlo con el método <code>equals()</code> . Con String deberemos utilizar equals.
Si el tipo de retorno de un método es un tipo primitivo, se devolverá el valor almacenado por el tipo primitivo .	Si el tipo de retorno de un método es de tipo de referencia, se devolverá la referencia de un objeto . Esto implica que, si en ese método se crea un objeto (variable local a ese método) este podría sobrevivir incluso después de finalizar el método, ya que se está devolviendo una referencia a dicho objeto presente en el montículo (HEAP) como retorno del método. (lo veremos en un patrón <i>factory method</i> más adelante)

4. Arrays en Java

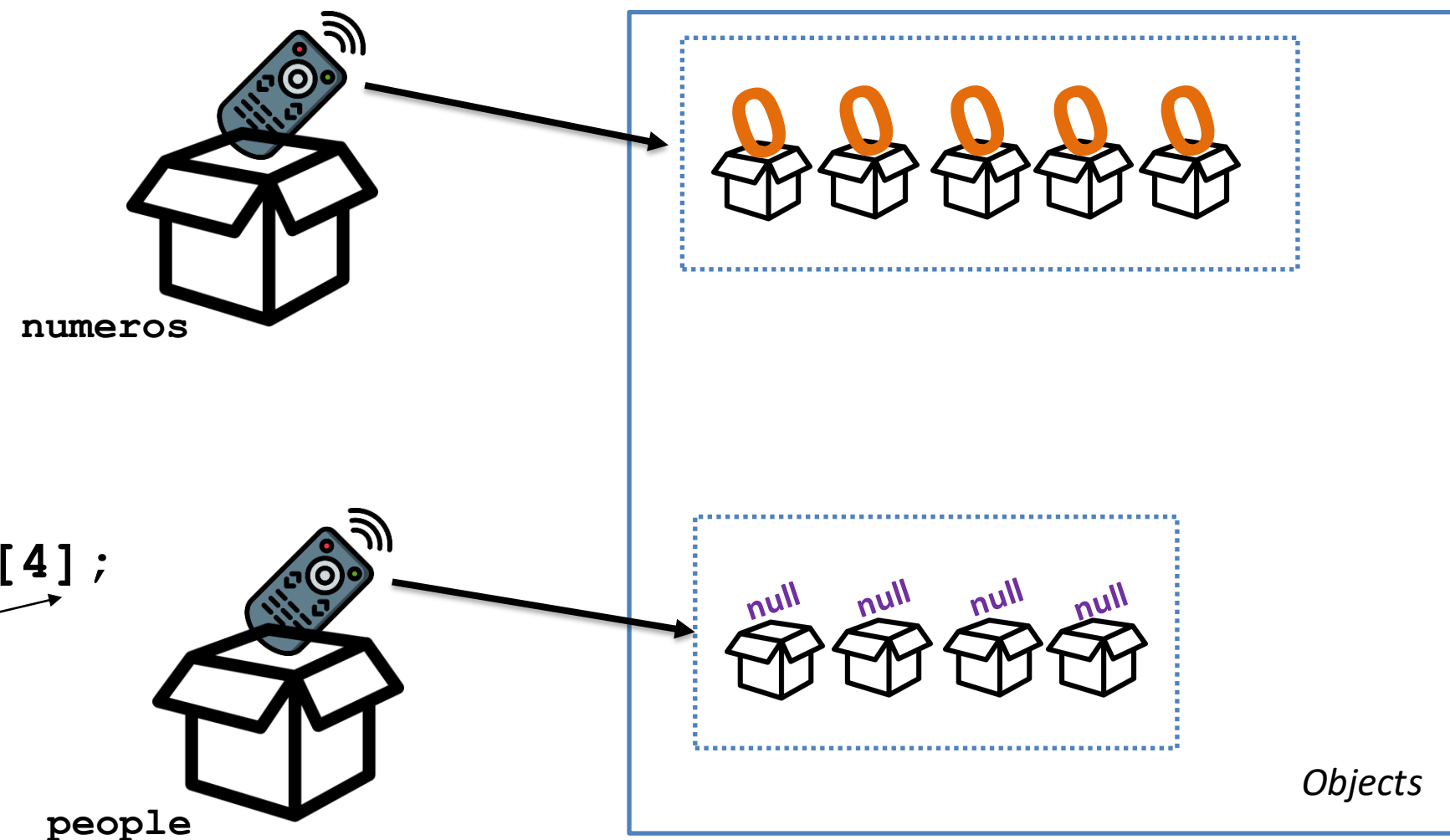
Array en Java: un array, es un grupo de variables (llamadas **elementos** o componentes) conteniendo valores que son **todos del mismo tipo** (ya sean primitivos o de referencia). Como se ha visto antes, un **Array es un objeto**, por lo tanto, **está considerado un tipo de referencia**.

Creación: ya que se trata de un objeto será necesario utilizar new y se obtendrá una referencia a dicho objeto.

```
int[] numeros = new int[5];
```

```
Person[] people = new Person[4];
```

¡No lleva paréntesis!
¡Ojo!



4. Arrays en Java

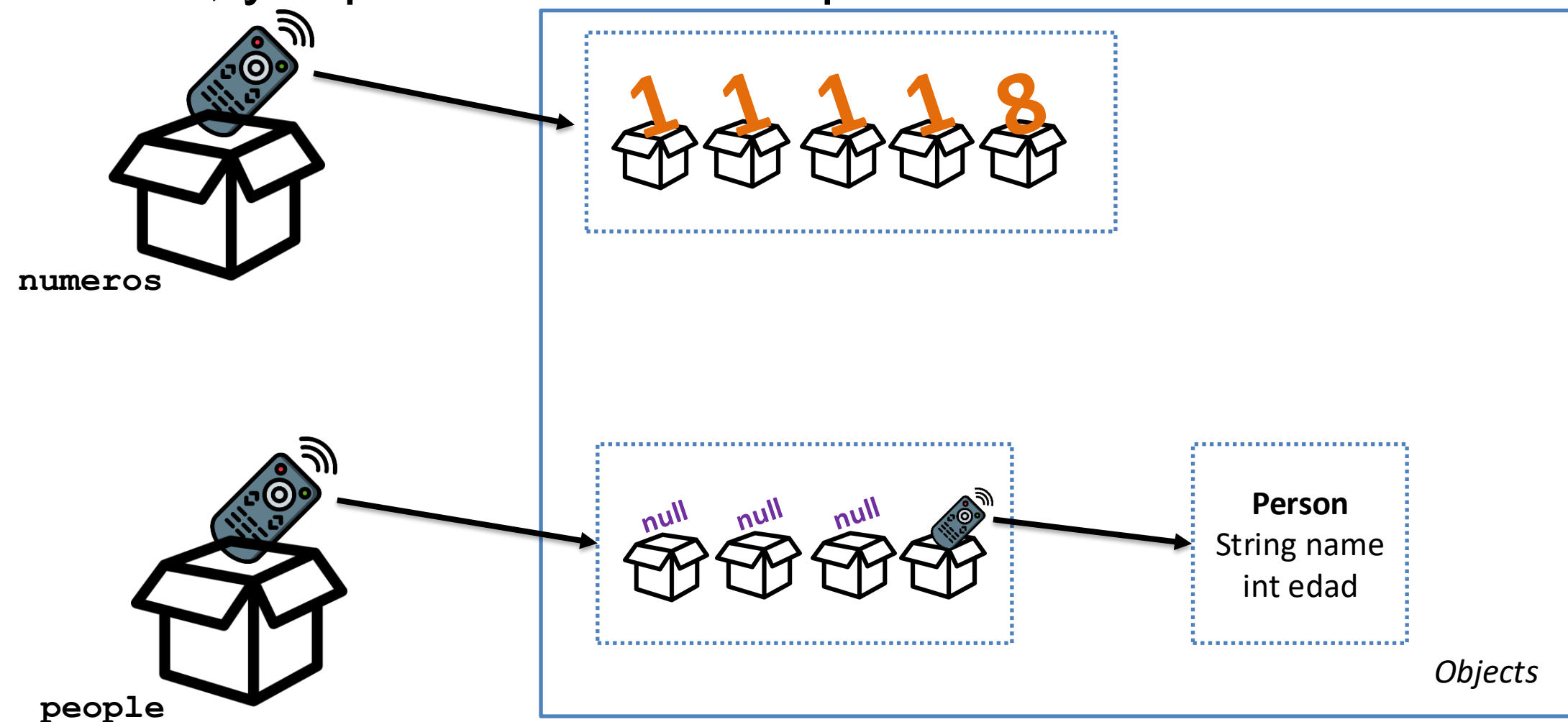
Inicialización: como se observa en la imagen anterior, se han creado Arrays pero no se han inicializado sus elementos. En el caso de ser de tipo primitivo tendrán su valor por defecto y en el caso de tipo de referencia serán null. Es posible inicializar los Arrays con valores entre llaves.

```
int[] numeros = {1,1,1,1,1};
```

Acceso: para acceder a cada uno de los elementos bastará utilizar corchetes y el índice apropiado. Debemos asegurarnos de que tengan valor, ya que de otro modo podríamos obtener una excepción accediendo a una referencia nula.

```
int[] numeros = {1,1,1,1,1}  
numeros[4] = 8;
```

```
Person[] people = new Person[4];  
people[3] = new Person();
```



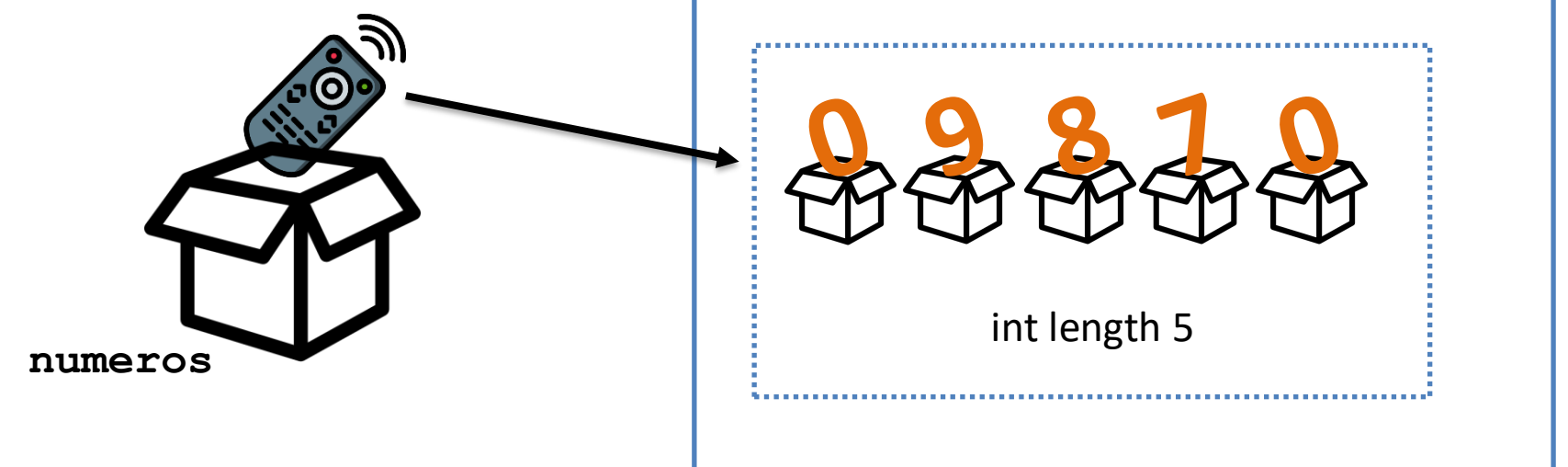
4. Arrays en Java

Atributo length: recordad que los Arrays en Java son objetos y entre los atributos de un array nos encontramos uno muy útil llamado **length** que permite obtener el número de elementos del array (independientemente de si están inicializados o no).

```
final int ARRAY_LENGTH = 5;
int[] numeros = new int[ARRAY_LENGTH];
numeros[1] = 9;
numeros[2] = 8;
numeros[3] = 7;
int total = 0;
```

```
for (int counter = 0; counter < numeros.length; counter++) {
    total += numeros[counter];
}
```

```
System.out.printf("Total: %d%n", total);
```



4. Arrays en Java

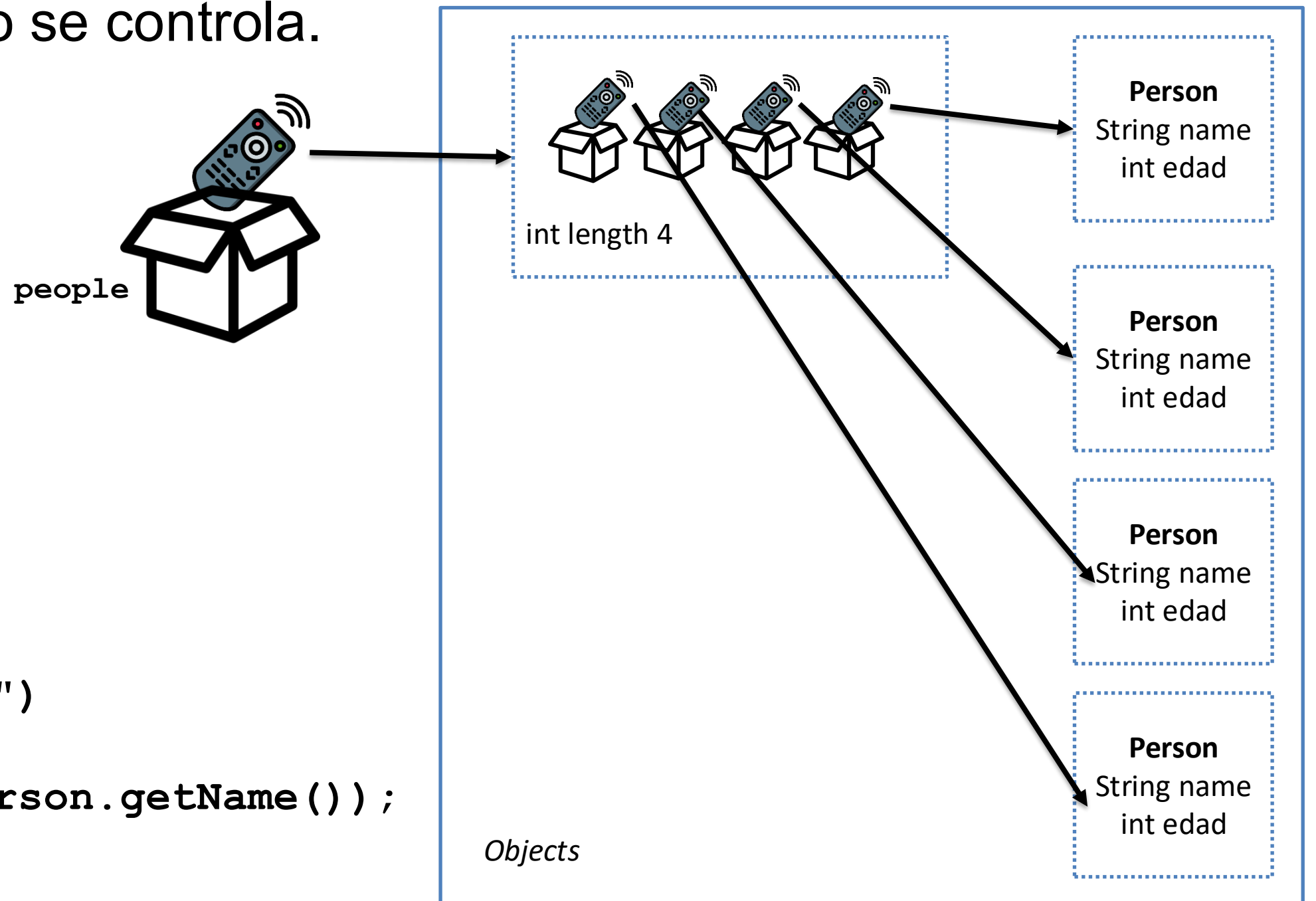
Sintaxis de foreach en java: es posible emplear un nuevo for (denominado en otros lenguajes foreach) con Arrays o colecciones como veremos más adelante. Su sintaxis se puede observar en el siguiente ejemplo. El principal problema que nos podemos encontrar es que uno de los elementos del array sea `null`, produciéndose una excepción si no se controla.

```
final int AMOUNT_OF_PEOPLE = 4;

Person[] people = new Person[AMOUNT_OF_PEOPLE];

// Create multiple people with a classical for
for (int i = 0; i < people.length; i++) {
    people[i] = new Person("Person"+i, 18+i);
}

// Iterate over the Array (new for called "foreach")
for (Person person : people) {
    System.out.printf("Hola, mi nombre es %s%n", person.getName());
}
```



Biblioteca.jar: Métodos útiles en OpMat.java

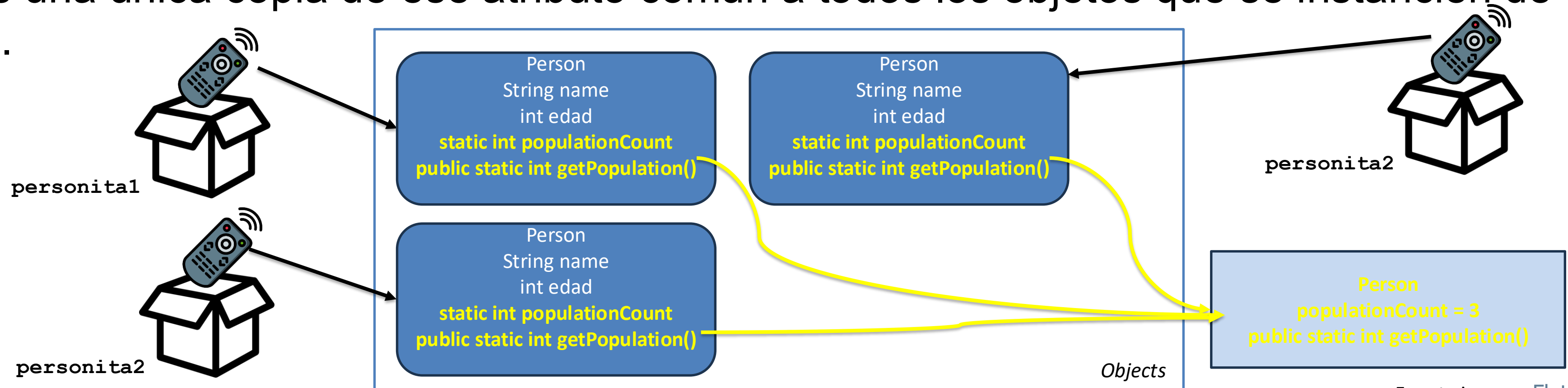
PACKAGE	CLASS	TREE	DEPRECATED	INDEX	HELP
ALL CLASSES					
SUMMARY: NESTED FIELD CONSTR METHOD			DETAIL: FIELD CONSTR METHOD		
SEARCH: Search					
static float[][]	importFromDisk (java.io.File f, java.lang.String delimiter, float dummy)	This method imports a float[][] from a delimited file			
static int[][]	importFromDisk (java.io.File f, java.lang.String delimiter, int dummy)	This method imports an int[][] from a delimited file			
static void	inputMat (double[][] matrix)	This method reads from the keyboard a double[][]			
static void	inputMat (float[][] matrix)	This method reads from the keyboard a float[][]			
static void	inputMat (int[][] matrix)	This method reads from the keyboard an int[][]			
static void	inputMat (java.lang.String[][] matrix)	This method reads from the keyboard a String[][]			
static java.lang.String[]	join (java.lang.String[][] a, java.lang.String[] b)	This method returns the join of a String[][] and a String[].			
static java.lang.String[]	join (java.lang.String[][] a, java.lang.String[][] b)	This method returns the join of its two String[][] arguments.			
static double[][]	loadFromDisk (java.io.File f, double dummy)	This method reads from disk an double[][] written with binary format			
static float[][]	loadFromDisk (java.io.File f, float dummy)	This method reads from disk an float[][] written with binary format			
static int[][]	loadFromDisk (java.io.File f, int dummy)	This method reads from disk an int[][] written with binary format			
static void	printToScreen (double[][] matrix)	This method prints to screen a double[][] with fixed (8.3) column and decimal widths			
static void	printToScreen (float[][] matrix)	This method prints to screen a float[][] with fixed (8.3) column and decimal widths			
static void	printToScreen (int[][] matrix)	This method prints to screen an int[][] with fixed (8) column widths			
static void	printToScreen (java.lang.String[][] matrix)	This method prints to screen a String[][] with fixed column widths (20)			
static void	printToScreen2 (java.lang.String[][] matrix)	This method prints to screen a String[][] with minimum (calculated but equal) column widths			
static void	printToScreen3 (java.lang.String[][] matrix)	This method prints to screen a String[][] with minimum and possibly different widths for each column			

Métodos interesantes para mostrar una tabla de valores (sin cabecera)

5. Atributo/Método de instancia o de clase

Los atributos y métodos definidos en una clase **SIN** el modificador `static` se denominan métodos o atributos de instancia. Esto es debido a que cada objeto tiene una copia separada de todos los atributos en su instancia.

Los atributos y métodos definidos en una clase **CON** el modificador `static` se denominan métodos o atributos de clase. El modificador `static` provoca que un **atributo/método pase a estar compartido por todos los objetos de esa clase**. Solo existe una única copia de ese atributo común a todos los objetos que se instancien de esa clase.

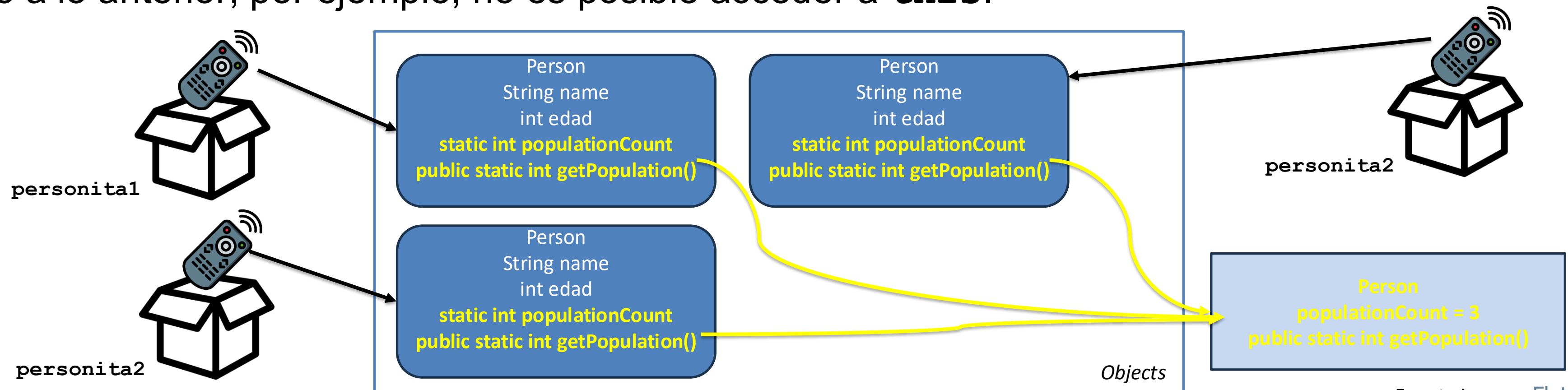


5. Atributo/Método de instancia o de clase

En el caso de los atributos o métodos de clase, **existen, aunque no se haya creado ningún objeto de su clase, por lo que se puede acceder sin crear un objeto**. Para acceder a métodos o atributos de clase se utiliza el **nombre de la clase seguido del punto**.

```
Person.populationCount = 3  
int population = Person.getPopulation();
```

Importante (Ámbito): desde un método estático solo se puede acceder a atributos estáticos. Esto es debido a lo anterior, por ejemplo, no es posible acceder a **this**.



Métodos estáticos en Esdia.java

Method Summary

All Methods	Static Methods	Concrete Methods
Modifier and Type	Method	Description
static String	getVersion()	Devuelve la versión actual de la biblioteca.
static boolean	isValidPath(String possiblePath)	Comprueba si una cadena de texto representa una sintaxis de ruta válida para el sistema operativo.
static void	printVersion()	Muestra en la salida estándar la información de versión y atribución de la biblioteca.
static double	readDouble(String prompt)	Muestra un mensaje y solicita un número real en coma flotante de doble precisión (double).
static double	readDouble(String prompt, double min, double max)	Muestra un mensaje y solicita un número double dentro de un rango cerrado [min, max].
static float	readFloat(String prompt)	Muestra un mensaje y solicita un número real en coma flotante de precisión simple (float).
static float	readFloat(String prompt, float min, float max)	Muestra un mensaje y solicita un número float dentro de un rango cerrado [min, max].
static int	readInt(String prompt)	Muestra un mensaje y solicita un número entero (int), repitiendo la petición hasta que la entrada sea válida.
static int	readInt(String prompt, int min, int max)	Muestra un mensaje y solicita un número entero dentro de un rango cerrado [min, max].
static long	readLong(String prompt)	Muestra un mensaje y solicita un número entero largo (long).
static Path	readPathFromKeyboard(String prompt)	Solicita una ruta por teclado y garantiza que sea sintácticamente válida antes de devolverla.
static String	readString(String prompt)	Muestra un mensaje en consola y lee una línea de texto completa introducida por el usuario.
static String	readString(String prompt, String[] options)	Solicita al usuario seleccionar una opción de entre un conjunto de opciones permitidas.
static String	readString(String prompt, String op1, String op2)	Solicita al usuario elegir entre dos opciones de texto válidas.
static String	readString_ne(String prompt)	Muestra un mensaje y lee una cadena de texto, garantizando que no esté vacía ni contenga únicamente espacios en blanco.
static boolean	si0no(String prompt)	Solicita una confirmación booleana en español, admitiendo exclusivamente [s/S/n/N].
static String	underline(String text)	Genera una cadena que contiene el texto recibido subrayado con caracteres '='.
static void	underline2(String text)	Imprime en la salida estándar el texto recibido subrayado con caracteres '='.
static boolean	yesOrNo(String prompt)	Solicita una confirmación booleana en inglés, admitiendo exclusivamente [y/Y/n/N].

Todos son **métodos estáticos**, por lo que no es necesario instanciar un objeto de dicha clase para poder utilizarlos. Pertenecen a la clase, no al objeto, por eso se les suele llamar también métodos de clase.

import vs import static

1. `import` se emplea para importar clases de un paquete concreto
2. `import static` se emplea para importar miembros estáticos (métodos/constantes) para usarlos sin prefijo de clase (es posible llamar al método estático directamente).

```
import static java.lang.Math.*;  
double r = sqrt(9);
```


6. Cambio de paradigma

Paradigma Orientación a Objetos

¿Qué es un paradigma de programación?

Los paradigmas de programación se pueden definir como una abstracción de las **características de los lenguajes y prácticas empleadas en ellos**, pero no se trata de los lenguajes en sí mismos.

Algunos paradigmas populares:

- Imperativo
- Declarativo
- **Estructurado**
- Procedural
- **Orientado a objetos**
- Funcional
- Lógico
- Dirigido a eventos

Usted viene
de aquí



Usted ahora
está aquí

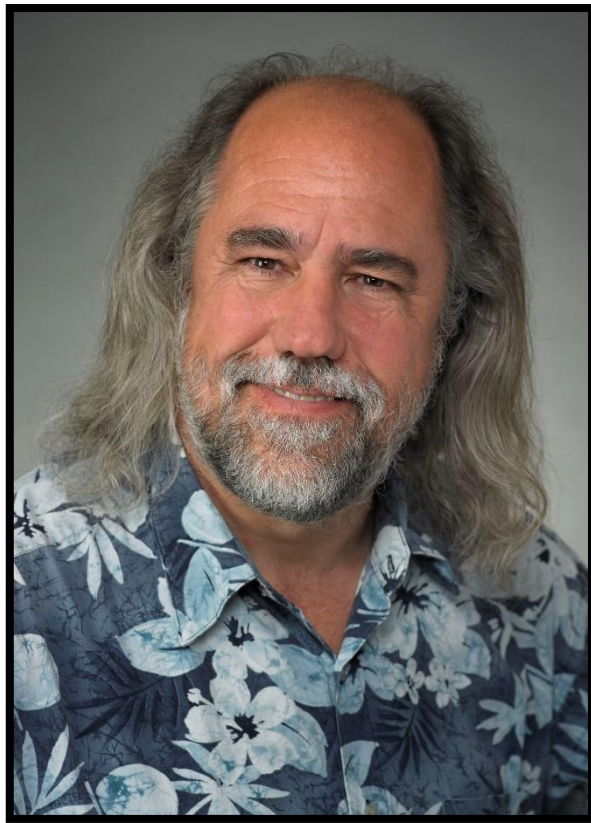


Cambio de paradigma
Por lo tanto, habrá otras
características y otras
prácticas.



Casi todos los lenguajes ofrecen características que **pueden clasificarse bajo múltiples definiciones de paradigma**. Por ejemplo, Java permite tanto la creación de objetos como el uso de *lambdas*. Estas dos características son comunes a la programación **orientada a objetos y a la funcional**. Se habla de lenguajes multi-paradigma.

Orientación a Objetos



Grady Booch

“La programación orientada a objetos es un método de implementación en el que los programas se organizan como colecciones cooperativas de objetos, cada uno de los cuales representa una instancia de alguna clase, y cuyas clases son todas miembros de una jerarquía...”

Un **Modelo Objeto** es un marco de referencia conceptual, en el que se establece el conjunto básico de los conceptos, la terminología asociada y el modelo de computación de los sistemas software soportados por la tecnología orientada a objetos. En otras palabras, es un lenguaje común que utilizar con otros miembros del equipo de desarrollo para usar este paradigma y aprovecharlo al máximo.



Puede resultar complicado al principio, se tratará durante el curso...

*“un Modelo Objeto está formado por cuatro elementos fundamentales: abstracción, encapsulamiento, modularidad y jerarquía; y por tres elementos secundarios: *tipos, concurrencia y persistencia*.”*

Clases y objetos en Java: autoevaluación



Revisa estos conceptos importantes que se deben conocer hasta la fecha:

- **Paquete:** nomenclatura habitual (*modularidad*)
- **Clase:** ¿cómo definir una clase? (*abstracción*)
- **Objeto:** ¿cómo instanciar objetos de una clase? `new MyClass()`
- **Atributos:** de instancia o de clase (`static`)
- **Métodos:** de instancia o de clase (`static`)
- **Modificadores de visibilidad aplicados a atributos y métodos.**
- **Atributo vs Variable local** en un método
- `this`: referencia a la propia instancia
- **Constructor:** ctor por defecto ¿Cuándo se ejecuta? ¿Puede haber varios? (*encapsulación*)
- **Getters y Setters:** ¿Para qué sirven? ¿Por qué existen? ¿Cómo generarlos? (*encapsulación*)
- **Organización de código en una clase:** convenciones de orden atributos, métodos, *getters* y *setters*, etc.
- **Tamaño de una clase:** Intentad mantener las clases pequeñas ≈ 150 líneas.
([Clean code capítulo 10](#) ↘ 🖱)
- **Comentarios:** para generar documentación de una clase ([JavaDoc](#)) (los necesarios)

Clases y objetos en Java: autoevaluación



Revisa estos conceptos importantes que se deben conocer hasta la fecha:

- **this**: comprobad que ocurre con una variable local a un método que tiene el mismo nombre que un atributo de la clase (esto pasa cuando generáis los *setters*).
- Cread una clase con diferentes constructores y cread varios **objetos** de esa clase con **distintos constructores. Comprobad con el depurador sus valores poniendo un punto de parada.**
- **Cread un atributo estático (static) en una clase** y cambiad su valor en diferentes objetos. ¿Qué ocurre?
- Cread un método estático en una clase y tratad de modificar un atributo de instancia ¿Qué ocurre?
- **Utilizad el depurador para ver el orden de ejecución** de las sentencias (asignaciones en los atributos, constructor, etc.) Conoced el flujo de ejecución **en la creación de un objeto (asignaciones en los atributos, ejecución del constructor)**
- Cread **varios métodos** en una misma clase **con el mismo nombre y firma** (mismo tipo de argumentos). Ahora cambiad el **tipo** de argumentos. ¿Qué ocurre?
- Pasad un objeto (**tipo referencia**) como parámetro a un método y cambiad su valor dentro de este. ¿Qué ocurre? ¿Tras la llamada ha cambiado su valor?
- Pasad un **tipo primitivo** como parámetro a un método y cambiad su valor. ¿Qué ocurre? ¿Tras la llamada ha cambiado su valor?
- ¿Por qué los atributos de una clase son normalmente privados y los *getters/setters* públicos? (*encapsulación*)

Clases y objetos en Java

Herramienta para aprender visualizando lo que ocurre en la ejecución del código:

<https://pythontutor.com/java.html#mode=edit> (solo para ejemplos sencillos)

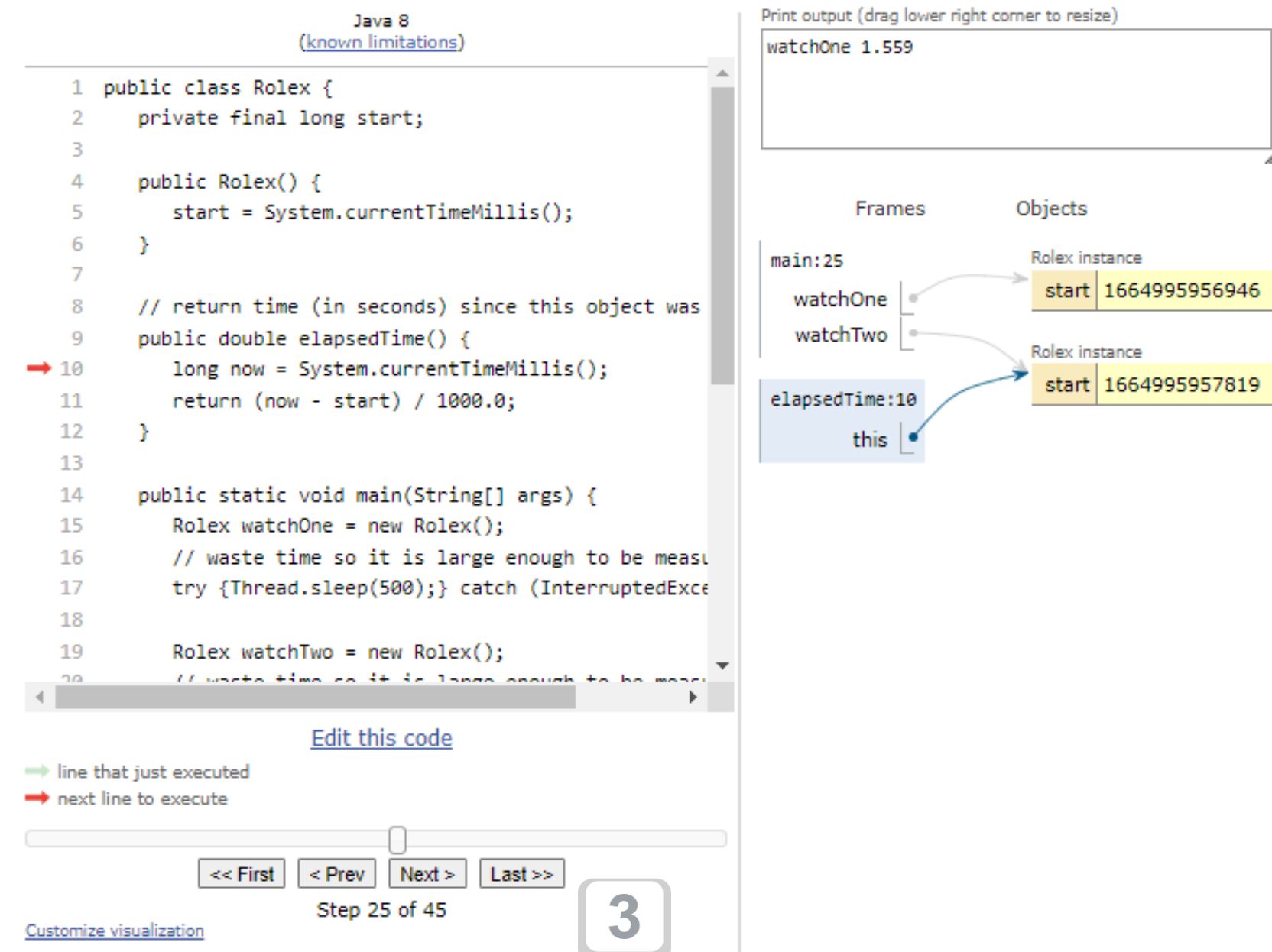
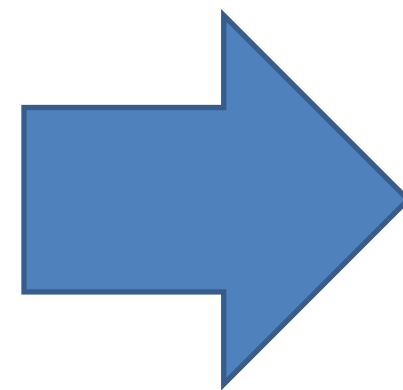
Java coding tutor - Learn Java by visualizing code execution (also visualize [Python](#), [JavaScript](#), [C](#), and [C++](#) code)

Write code in Java 8

```
1 public class Rolex {
2     private final long start;
3
4     public Rolex() {
5         start = System.currentTimeMillis();
6     }
7
8     // return time (in seconds) since this object was created
9     public double elapsedTime() {
10        long now = System.currentTimeMillis();
11        return (now - start) / 1000.0;
12    }
13
14    public static void main(String[] args) {
15        Rolex watchOne = new Rolex();
16        // waste time so it is large enough to be measurable
17        try {Thread.sleep(500);} catch (InterruptedException e) {}
18
19        Rolex watchTwo = new Rolex();
20        // waste time so it is large enough to be measurable
21        try {Thread.sleep(500);} catch (InterruptedException e) {}
22
23        // right now, watchOne is older
24        System.out.println("watchOne " + watchOne.elapsedTime());
25        System.out.println("watchTwo " + watchTwo.elapsedTime());
26
27        Rolex watchTmp = watchOne;
28        watchOne = watchTwo;
29        watchTwo = watchTmp;
30
31        // swapped! now watchTwo is the older one
32        // e.g. watchTwo.elapsedTime() returns a value
33        // (slightly) larger than watchOne.elapsedTime()
34        System.out.println("watchOne " + watchOne.elapsedTime());
35        System.out.println("watchTwo " + watchTwo.elapsedTime());
36    }
37 }
```

Visualize Execution

render all objects on the heap (Python/Java) draw pointers as arrows [default]



4

3

7. Práctica: ejercicios

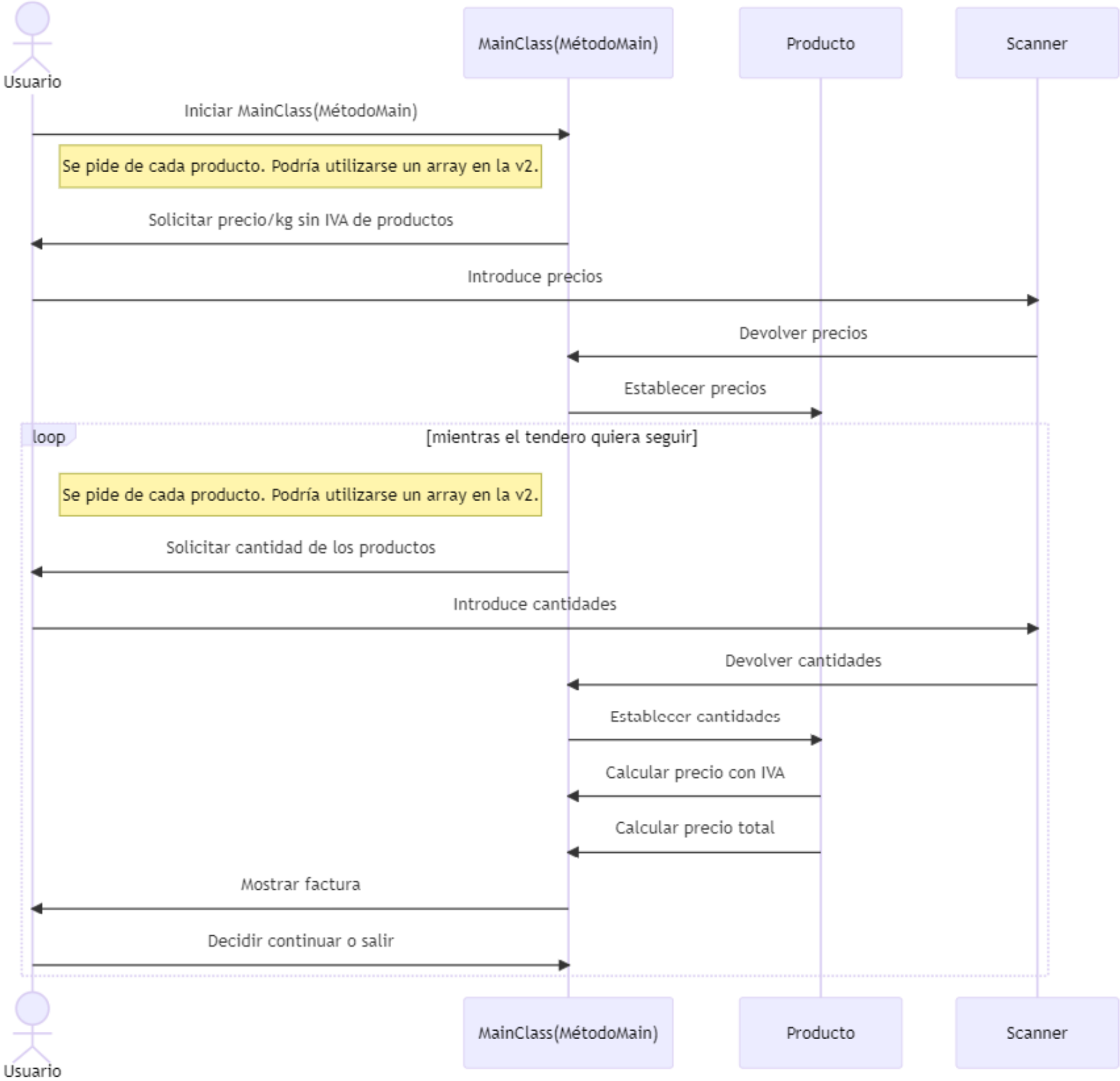
¡Ejercicios!

Subid los resultados a
vuestro repositorio

Realizad los ejercicios de
forma autónoma

Ejercicio Frutería

Posible enfoque inicial (simple)



Una frutería precisa de un programa que calcule la factura de la compra de sus clientes.

La frutería sólo trabaja con dos productos: peras y manzanas cuyo valor varía cada día.

El programa pedirá por consola al tendero el precio/kg sin IVA de las peras y las manzanas cada mañana al comienzo de su jornada laboral.

El IVA ya está prefijado en el programa.

Posteriormente el programa permite introducir la cantidad de cada tipo de fruta que desea comprar cada cliente, y calculará el precio total de la factura para dicho cliente, mostrando en forma de tabla el tipo de fruta, cantidad en Kg, precio con IVA por Kg, precio total con IVA por fruta.

Prueba a utilizar los métodos adecuados de Esdia.java de Biblioteca.jar en lugar de la clase Scanner.

Utiliza Arrays para almacenar los productos y recórrelos cuando sea necesario para acceder a su información.

Trabajo individual en casa

Completad los ejercicios, apuntad las
dudas para preguntarlas

¡Para l@s curios@s! 🧐

¿Sabrías definir y poner un ejemplo (refiriéndote a código Java) de los siguientes conceptos?

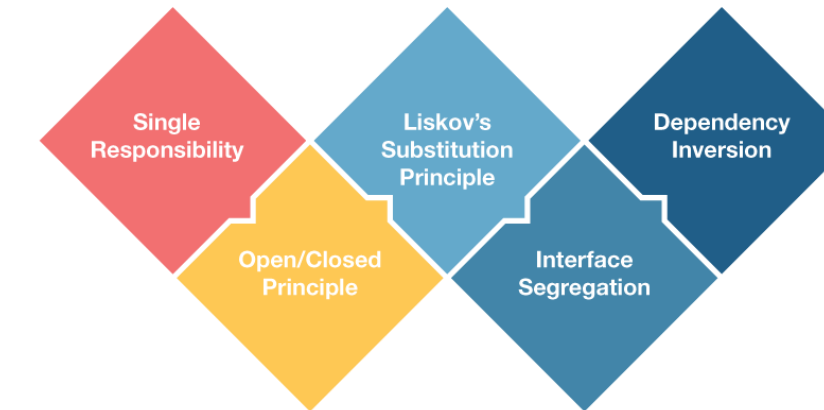
- Abstracción
- Encapsulación
- Modularidad
- Jerarquía: de herencia y composición. (Esto lo veremos más adelante)

¿A qué corresponden las siglas?
SOLID, DRY, KISS y TDD
¡Búscalas!

¿Qué es Clean Code 🖱️?

Se ve mucho en ofertas de empleo...

S.O.L.I.D.



Java Software Developer Spain (Remote) [Easy Apply](#) [Save](#) ...

What Are We Looking For?:

- Great technical mindset to work in an environment where we promote platforms, **clean code**, **design principles (SOLID, KISS, DRY)**, and TDD.
- Strong design pattern skills and domain driven design.
- Passionate about new challenges and worried about being up to date.
- **At least 4 years of experience with Core Java / J2EE.**
- Strong understanding of Data Structures & Algorithms.
- At least 4 years of experience in:
 - Java, Spring, Hibernate, Microservices
 - Spring-related technologies such as Spring Boot, Spring Cloud.
 - Application support, maintenance, and analysis.
 - software development life cycle & should have the ability to read logs.
- At least 4 years of experience in Project life cycle activities on development and maintenance projects.
- Traditional relational databases like MySQL, no SQL such as Elasticache, etc
- Knowledge of CI/CD Pipelines built with BitBucket, Maven, Jenkins
- Experience with docker.
- **Experience with git.**

Java Developer Spain (Remote) [Apply](#) [Save](#) ...

¿Qué valoramos?

- Experiencia en el desarrollo de aplicaciones web en Java con Spring Boot.
- **Desarrollo de código limpio.**
- Testing de aplicaciones.
- Conocimientos de metodologías ágiles (SCRUM).

Además, aunque no es imprescindible, también nos gustaría:

- Experiencia en desarrollo con otros lenguajes (Net Core, Kotlin, Nodejs, Python)
- Microservicios.
- Dialog flow.
- Cloud Platforms (Azure, AWS, GCP).
- Kafka.
- Inglés.

Java Developer Spain (Remote) [Easy Apply](#) [Save](#) ...

We're looking for someone who has

- At least 3+ Years of experience in software development.
- **Passion for software engineering and craftsman-like coding prowess.**
- **Extensive knowledge of object-oriented programming**, algorithms and software architecture
- Back-end: significant experience with Java and knowledge of its strengths and weaknesses.
- Experience exposing APIs.
- Databases: a solid understanding of SQL and good working knowledge of PostgreSQL and more unconventional databases (Redis, MongoDB).
- **Solid knowledge of Git**, Linux environments and the command line.

Nice to have

- Knowledge of any of the next frameworks (Spring Boot, Vert.x)
- Experience with virtualization and orchestration tools (Docker, Docker-compose, Docker swarm, Kubernetes)

Java Developer Spain (Remote) [Apply](#) [Save](#) ...

programador en este lenguaje para unirse a nuestro equipo de desarrollo en proyectos del sector retail, trabajando en las herramientas de gestión del producto y el catálogo de e-commerce.

Para eso, necesitamos que:

- Desarrolles en Java 8 o superior. En esencia, esto es lo más importante.
- Tengas experiencia con algunos módulos de Spring, como Boot, Data o Security.
- Tengas experiencia en web services.
- Tengas soltura manejando bases de datos tanto relacionales como no relacionales.
- Testing de pruebas unitarias, BDD y TDD.
- Sepas enfrentarte a un entorno de API Rest, trabajamos en proyectos basados en arquitectura de microservicios.

Eso sí, si partes de una base sólida desarrollando en Java y tienes una buena capacidad de análisis, estamos seguros de que encajarás a la perfección.

Y sería genial que también:

- Te preocupases por hacer aplicaciones web sólidas y con buen rendimiento.
- **Tengas conocimiento en CI/CD**
- **Trabajases bajo los principios de Clean Code.**
- Hubieras trabajado bajo metodologías ágiles. ¡Nos gusta un *sprint* más que a un tonto un lápiz!